



Technische
Universität
Braunschweig

BACHELOR THESIS

Write down your Title right here!

Fabian Alich

5310894

Algorithmik

Institute of Operating Systems and Computer Networks
Algorithms Division

First reviewer

(Anon Anonymous, Institute of XY)

Second reviewer

(Anon Anonymous, Institute of YX)

Supervised by

(Anon Anonymous)

(tba)

June 12, 2025

Statement of Originality

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, June 12, 2025

Aufgabenstellung

This is where your “Aufgabenstellung” goes. You should get this from your supervisor!

Abstract

Your abstract gives a short introduction to your thesis (context), the problem description, and a brief description of your results.

You may want to include a german abstract, too.

Contents

1	Introduction	1
1.1	Results	1
1.2	Related Work	1
1.2.1	Delaunay Triangulations	1
1.2.2	Flips in Triangulations	2
1.2.3	degree constrained minimum spanning tree	3
1.3	Overview	7
2	Preliminaries	9
3	Content	11
3.1	Instanzen Generation	11
3.1.1	Ja - Instanzen	11
3.1.2	Nein - Instanzen	11
3.2	Heuristiken	12
3.2.1	Flips	12
3.2.2	OrTools mit Zielfunktion	13
3.3	Solver	13
3.3.1	OrTools	13
3.3.2	SAT	13
4	Auswertung	15
4.1	Versuchsumgebung	15
4.2	Heuristiken	15
4.2.1	Instanzen und Laufzeit	15
4.2.2	Flips	15
4.2.3	OrTools mit Zielfunktion	15
4.3	Solver	15
4.3.1	Instanzen und Laufzeit	15
4.3.2	OrTools	15
4.3.3	Sat	15
5	Conclusion (and Future Work)	17

List of Figures

1.1	<i>A perspective view of a terrain</i> aus [4, p. 183]	2
1.2	<i>Examples of edge move, edge rotation, and edge slide</i> aus [1, p. 70]	3

List of Tables

1 Introduction

You can write an introduction here!

1.1 Results

We proof that the TRAVELING SALESMEN PROBLEM(TSP) is NP-complete.

... that the \textsc{Traveling Salesmen Problem}(TSP) is ...

You can use abbreviations for your problem names to avoid redundancy. Try to use existing abbreviations from related work if possible.

Formel zwischen lagern:

1.2 Related Work

In diesem Abschnitt werden verschiedene wissenschaftliche Arbeiten aus diesem Themenbereich betrachtet. Dies dient dazu, einen Überblick über den aktuellen Entwicklungsstand zu erhalten. Dazu werden drei Bereiche begutachtet. Der erste Bereich behandelt die *Delaunay Triangulation*. Diese zählt zu den bekanntesten Triangulations und bietet einen grundlegenden Einblick in das Thema Triangulation. Der zweite Bereich umfasst *Flips in Triangulations*. Hier wird untersucht, wie sich Triangulation durch lokale Änderungen gezielt modifizieren lassen. Der dritte und letzte Bereich befasst sich mit dem Problem des *degree constrained minimum spanning tree*. Dabei werden insbesondere Ansätze zur Einhaltung von Grad Beschränkungen betrachtet.

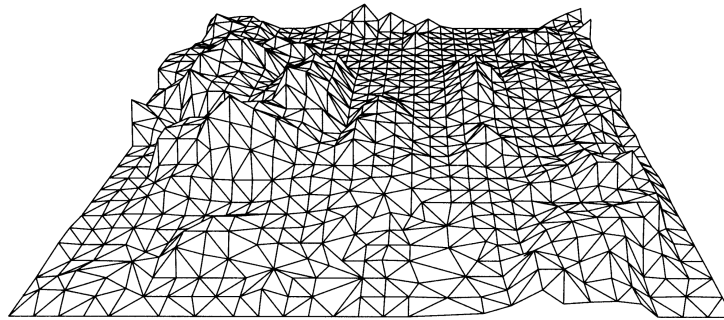
Es ist noch zu erwähnen, dass Sharir et. al. [11] den minimalen und maximalen Grad in einer Triangulation untersucht haben. Darüber hinaus haben Datta et. al. [2] und Gewali et. al. [6] sich mit Triangulation beschäftigt, bei denen jeder Knoten einen geraden Grad hat. Da diese beiden Themenbereiche keinen direkten Mehrwert für die vorliegende Arbeit bieten, werden sie hier lediglich der Vollständigkeitshalber namentlich erwähnt.

1.2.1 Delaunay Triangulations

Als Grundlage für den Algorithmus kann die *Delaunay-Triangulation* betrachtet werden. Diese liefert eine Triangulation, die möglichst gleichschenklige und gleichwinklige Dreiecke verwendet [10]. Dies wird erreicht, indem der minimale Winkel maximiert wird. Im Buch von de Berg et al. [4] wird diese Variante der Triangulation ausführlich behandelt.

Triangulations eignen sich durch ihre Struktur besonders gut zur Darstellung von Höhendaten im zweidimensionalen Raum (siehe Figure 1.1). Dabei kann ausgenutzt werden, dass Dreiecke unterschiedliche Innenwinkel und Kantenlängen besitzen.

Figure 1.1: A perspective view of a terrain aus [4, p. 183]



Das Grundproblem besteht darin, dass die Höhendaten nur an bestimmten Punkten bekannt sind und die restlichen Werte daher geschätzt werden müssen. Dies kann durch die Delaunay-Triangulation gut gelöst werden, da die Dreiecke aufgrund ihrer gleichschenkligen Eigenschaft visuell geeignet sind, die fehlenden Daten zu ergänzen.

Es werden auch Formel für die Anzahl der Kanten und Dreiecke in einer Triangulation angegeben. Für eine Punktmenge mit n Punkten und k Punkten auf der konvexen Hülle gilt:

$$\text{Anzahl Kanten: } 3n - 3 - k$$

$$\text{Anzahl Dreiecke: } 2n - 2 - k$$

1.2.2 Flips in Triangulations

Flips sind Möglichkeiten, um Kanten in einer Triangulation auszutauschen. Wie oben erwähnt, müssen Triangulation eine feste Anzahl an Kanten besitzen [4]. Dementsprechend können keine Kanten hinzugefügt oder entfernt werden, um den Grad einer Triangulation zu verändern. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [7] zeigten, dass jede Triangulation mindestens

$$\frac{n - 4}{2}$$

flipbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut der wissenschaftliche Arbeit *Flips in planar graphs* [1] lässt sich jede mögliche Triangulation in jede beliebige andere Triangulation überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn eine Gradbeschränkte Triangulation möglich ist, diese mit den nötigen Flips konstruiert werden kann.

TODO überarbeiten **Sollte ich das später machen, hier die Sektion nennen**

Eine weitere Art von Flips sind sogenannte k -Flips. Diese stellen eine zusätzliche Möglichkeit dar, Kanten innerhalb einer Triangulation zu verschieben. Dabei werden k Kanten entfernt und durch k neue Kanten ersetzt. Normale Flips entsprechen somit

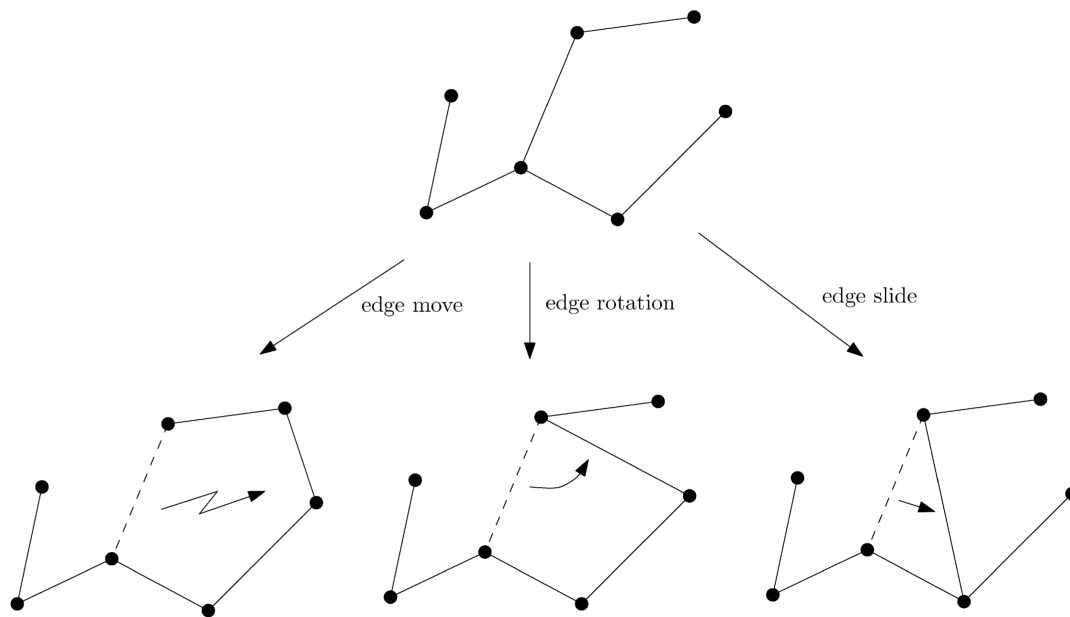


Figure 1.2: *Examples of edge move, edge rotation, and edge slide* aus [1, p. 70]

1-Flips. Der Vorteil von k -Flips liegt darin, dass mit einem Schritt größere strukturelle Veränderungen möglich sind.

Drei beispielhafte Flips sind in Figure 1.2 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.

1.2.3 degree constrained minimum spanning tree

Eine Gradbeschränkung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht, welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained minimum spanning tree* ist, dass ein Minimum Spanning Tree erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von d hat. Dabei wird d global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [8] vorgestellt, welche das Problem lösen sollen. Der erste Ansatz ignoriert zu Beginn den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt, welche die Gradbeschränkung nicht verletzen. Im weiteren Verlauf wird versucht, die Anzahl der Kanten zu minimieren.

Ein anderer Ansatz nutzt aus, dass der minimale Spannbaum ohne Grad Beschränkung in polynomialer Zeit gelöst werden kann. Dieser Ansatz erzeugt zuerst einen minimalen

1 Introduction

Spannbaum, welcher die Gradbeschränkung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Diese neuen Gewichte werden mit folgender Formel berechnet,

$$weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

wobei *weight* das aktuelle Gewicht der Kante ist. Die Variable *fault* hat den Wert, $\{0, 1, 2\}$ abhängig davon, wie viele Knoten an der Kante gerade die Gradbeschränkung verletzen. Die Variablen *min* und *max* bezeichnen das kleinste beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler Spannbaum gebildet, wobei durch das neue Gewicht nun Kanten bevorzugt werden, welche keine Gradbeschränkung verletzen. Diese beiden Schritte können so lange wiederholt werden, bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Das Problem bei diesem Ansatz ist, dass Lösungen fälschlicherweise als *nicht möglich* bewertet werden können aufgrund der maximalen Iterationen.

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst das Problem des *degree constrained minimum spanning tree* nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf die *Degree constrained Triangulation* angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt. Danach werden weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht. Bei diesen Knoten wird überprüft, ob sie gleich gute oder bessere Ergebnisse liefern. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines *greedy*-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung der lokalen Problematik stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Punkt in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Punkt ausgewählt.

Der nächste Ansatz wird *simulated annealing* genannt. Auch hier werden nur neue Knoten gesucht, um bestehende Lösungen zu verbessern. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei ist $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist *i* und die mögliche neue Lösung ist *j*. Die Funktion *f* gibt hierbei die Kosten der Lösung an. Der Parameter *c* wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Die Veränderung von *c* bewirkt die gewünschte Änderungsrate. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$.

Der letzte Ansatz wird erst am Ende vorgestellt und beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet. Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

Krishnamoorthy et.al. [9] hat Evolutionsalgorithmen noch mal genauer betrachtet. Der erste Schritt ist es eine Codierung zu wählen, welche nur *richtige* Spannbäume darstellen kann. Sonst kann es passieren, dass von den erstellten Lösungen viele nicht genutzt werden können und somit nicht relevant sind. Wenn eine passende Codierung gewählt wurde, können dann nur noch der Grad und das Gewicht betrachtet werden.

Eine Möglichkeit wie Mutationen stattfinden können, ist die *Crossover* Methode. Dabei existieren zwei *Eltern* Lösungen P_1 und P_2 . Aus diesen werden

$$2 \cdot (n - 2)$$

verschiedene *Kinder* erzeugt, da es $n - 1$ Veränderungsmöglichkeiten gibt. Anschließend werden die *Kinder* bewertet und sortiert. Das beste *Kind*, das P_1 am ähnlichsten ist, wird ausgewählt. Ist dieses *Kind* besser als P_1 , ersetzt es P_1 . Für P_2 wird ein *Kind* nach dem gleichen Verfahren ausgewählt.

Eine weitere Möglichkeit ist die *Breeding* Methode. Dabei existiert ein Pool aus Lösungen. Zufällig werden Paare gebildet, die jeweils zwei *Kinder* erzeugen. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, dass alle möglichen Veränderungen berücksichtigt werden.

Beide Methoden sollen im Durchschnitt alle 10 Generationen die beste Lösung mit allen anderen Lösungen verglichen haben. Als mögliche Iterationsanzahl wird folgende Formel verwendet,

$$\frac{50 \cdot n}{\bar{b}}$$

dabei bezeichnet $\bar{b}$ die durchschnittliche Gradbeschränkung. Diese Formel sorgt dafür, dass große Instanzen mehr Iterationen erhalten, während leichtere Instanzen (mit höherem Grad) schneller bearbeitet werden.

Sandor P. Fekete et al [5] betrachten ebenfalls das *degree constrained minimum spanning tree* problem. Hierbei wird ein *Flow based Algorithmus* verwendet. Da dieser Ansatz wahrscheinlich nicht gut auf Triangulations übertragbar ist, wird er hier nur kurz behandelt.

Im praktischen Teil des Papers werden zwei Algorithmen vorgestellt. Diese erhalten einen minimalen Spannbaum und verändern ihn so, dass er eine Gradbeschränkung einhält. Dabei wird darauf geachtet, dass sich das Gewicht des Spannbaums nicht stark verschlechtert. Für diese Algorithmen werden Hauptoperationen namens *Adoptions* eingeführt. Diese funktionieren ähnlich wie Flips nur für einen Spannbaum. Es handelt sich also um Operationen, die eine legale Veränderung bewirken. Die beiden Algorithmen liefern eine Liste von Adoptionen, welche die gewünschte Änderung erzeugen. Der erste Algorithmus nutzt eine fraktionale Lösung und eine Greedy Strategie, um die Adoptionen zu finden. Der andere Algorithmus beschränkt die betrachteten Kanten auf jene des

1 Introduction

gegebenen Spannbaums. So können die Algorithmen in polynomialer Zeit eine Lösung finden, die eine Gradbeschränkung einhält.

Eine Abwandlung des minimalen Spannbaums mit Gradbeschränkung wurde von Ana Maria de Almeida et al. [3] untersucht. Dabei erweitern sie das Problem um eine Funktion, welche für jeden Knoten einen Mindestgrad vorgibt. Dieses Problem wird als *Min-Degree Constrained Minimum Spanning Tree* bezeichnet.

Es wurde das *k-Dimensional Matching Problem* verwendet, um zu zeigen, dass das *Min-Degree Constrained Minimum Spanning Tree* Problem in NP liegt. Zu erwähnen ist, dass dies in der dritten Dimension nicht gezeigt wurde. Allerdings soll das Problem schwieriger sein als das *Degree Constrained Minimum Spanning Tree*, obwohl auch dieses in NP liegt.

Auch in diesem Fall wird das Problem mit einer *Flow based Formulierung* betrachtet. Dabei werden zwei Formulierungen angegeben. Eine gerichtete und eine ungerichtete Formulierung. Die gerichtete Formulierung soll hierbei effizienter Ergebnisse liefern. Die Formulierungen wurden mit einem *Branch and Bound* Algorithmus getestet. Dabei wurde festgestellt, dass es bei der gerichteten Formulierung relevant ist, welcher Knoten als Wurzelknoten ausgewählt wird. Es konnte jedoch kein einheitlich optimaler Wurzelknoten für die Formulierungen festgestellt werden. Des Weiteren wurde häufig festgestellt, dass die Relaxierung keine guten Schranken lieferte. Dies konnte damit erklärt werden, dass die zugehörige Formel für den minimalen Spannbaum ohne Gradbeschränkung gleiche Ergebnisse lieferte.

1.3 Overview

If needed, write down where to find everything.

2 Preliminaries

Definition 2.1 (Überschneidung). Zwei Kanten gelten als Schneidet wenn sie sich in der Ebene kreuzen. Im Folgendem werden mit der Funktion $I : (\mathbb{Z}^2, \mathbb{Z}^2) \rightarrow \{\mathbb{Z}^2\}$ alle Kanten angegeben, die die übergebene Kante schneiden.

Definition 2.2 (Planarer Graph). Ein Planarer Graph ist ein Graph ohne überschneiden Kanten. Das bedeutet man kann alle Punkte und Kanten darstellen ohne das sich diese überschneiden.

Definition 2.3 (Triangulation). Eine Triangulation einer Punktmenge $P \in \mathbb{Z}^2$ ist ein Maximaler Planarer Graph. Das bedeutet man kann keine weiteren Kanten hinzufügen ohne das sich diese schneiden.

Theorem 2.4. *TODOGehört Sowas hier hin? Soll ich sowas nennen*

Alle Flächen außer der Äußeren Fläche einer Triangulation bestehen aus Dreiecken. Der Beweis wird von Mark Berg et.al. geliefert[4, p. 185].

Definition 2.5 (Grad). Der Grad gibt die Anzahl der anliegenden Kanten an einem Knoten an. Im folgenden wird er durch die Funktion $\deg : \mathbb{Z}^2 \rightarrow \mathbb{Z}$ angegeben.

Definition 2.6 (Gradbeschränkte Triangulation). Die Gradbeschränkte Triangulation hat die gleichen Voraussetzungen wie eine Triangulation. Zusätzlich gibt es noch eine Funktion $\deg^* : \mathbb{Z}^2 \rightarrow \mathbb{Z}$. Diese gibt den Grad an welcher ein Knoten haben muss.

Definition 2.7 (Flipp). Eine Operation bei der eine Kante entfernt und eine andere Hinzugefügt wird. Hierbei dürfen nach der Operation die Triangulations Eigenschaften nicht verletzt sein. Ein Flip wird durch die Funktion $\text{flip} : (\mathbb{Z}^2, \mathbb{Z}^2) \rightarrow (\mathbb{Z}^2, \mathbb{Z}^2)$ definiert.

Definition 2.8 (Diagonale Flipp). Ein Diagonaler Flipp hat die gleichen Eigenschaft wie ein Flipp. Zusätzlich wird für die Kante (b, c) die leeren Dreiecke $\triangle abc$ und $\triangle bcd$ gesucht und (a, d) zurückgeben. Er wird durch die Funktion $\text{flip}^* : (\mathbb{Z}^2, \mathbb{Z}^2) \rightarrow (\mathbb{Z}^2, \mathbb{Z}^2)$ definiert.

3 Content

3.1 Instanzen Generation

3.1.1 Ja - Instanzen

- Random n Knoten Generieren
- Triangulieren
- Knoten mit Grad speichern

Delaunay mit Flips

- Delaunay mit scipy erstellen
- k mal random Kante Diagonal Flippen
 - alle Dreiecke finden an der kante
 - * Bei allen ausgehenden Kanten der beiden Knoten die Kanten finden welche einen gemeinsamen Knoten haben
 - Die beiden Dreiecke die leer sind
 - Die Kante durch eine neue Kante ersetzen welche aus den beiden Knoten der Dreiecke besteht welche nicht an der alten Kante waren

Random Kanten

- Alle möglichen Kanten erstellen
- Random Kanten auswählen
- wenn diese keine Vorhande schneiden hinzufügen

3.1.2 Nein - Instanzen

- Move Point liefert meistens nein Instanzen
- Zwei Sollgrade von zwei Punkten Tauschen

3.2 Heuristiken

- eine Triangulation erstellen
- den sollGrad möglichst richtig
- Werden mit folgender Formel bewertet

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (3.1)$$

$$e_i = \begin{cases} \frac{\deg^*(i) - \text{diff}_i}{\deg^*(i)} & \deg^*(i) \geq \text{diff}_i \\ 0 & \text{sonst} \end{cases} \quad (3.2)$$

$$Q_G = \sum_{i=0}^n \frac{e_i}{n} \quad (3.3)$$

Die Formel liefert eine Evaluation (Q_G) für einen Graphen(G) mit n Knoten. Hierbei gibt $\deg^*(i)$ den gewünschten Grad an der stelle i an. Dazu passend gibt $\deg(i)$ den aktuellen Grad an. In der Brechnung (3.1) wird die Difference zwischen dem gewünschten und aktuellen Grad berechnet. In (3.2) wird diese Abweichung Prozentual bewertet. Am Ende wird in (3.3) ein Durchschnitt über alle Prozentualen Werte gebildet.

3.2.1 Flips

- Diagonale Flips
- k mal Wiederholen
 - Alle Knoten durchgehen
 - wenn falscher Degree auswählen
 - alle ausgehenden Kanten berechnen
 - Kante Auswählen
 - * random Kante auswählen
 - * gucken ob ein Knoten der Kante falschen Degree hat
 - * ja ? zurückgeben
 - * nein ? zu 20% zurückgeben
 - * nächste kante wählen

Flipe Kante wie oben wenn Gradbeschränkte Triangulation erfolgreich abbrechen

3.2.2 OrTools mit Zielfunktion

3.3 Solver

3.3.1 OrTools

3.3.2 SAT

4 Auswertung

4.1 Versuchsumgebung

- Implementation details (e.g., libraries, third-party programs, ...)
- In which environment are you testing (e.g., system specifications)
- What are you testing? (e.g., runtime, space needed, fps, ...)
- What are the results? Show fancy figures!
- What does the results mean? (e.g., give a little discussion on your results)
- ...

4.2 Heuristiken

4.2.1 Instanzen und Laufzeit

4.2.2 Flips

4.2.3 OrTools mit Zielfunktion

4.3 Solver

4.3.1 Instanzen und Laufzeit

4.3.2 OrTools

4.3.3 Sat

5 Conclusion (and Future Work)

This is the end of your thesis! Give a summary of what you did.

You can also write down possible future work you or other people could look at.

Bibliography

- [1] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [2] Basudeb Datta and Subhojoy Gupta. Degree-regular triangulations of surfaces. *arXiv preprint arXiv:1711.01247*, 2017.
- [3] Ana Maria de Almeida, Pedro Martins, and Maurício C de Souza. Min-degree constrained minimum spanning tree problem: complexity, properties, and formulations. *International Transactions in Operational Research*, 19(3):323–352, 2012.
- [4] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [5] Sándor P Fekete, Samir Khuller, Monika Klemmstein, Balaji Raghavachari, and Neal Young. A network-flow technique for finding low-weight bounded-degree spanning trees. *Journal of Algorithms*, 24(2):310–324, 1997.
- [6] Laxmi P Gewali and Bhaikaji Gurung. Degree aware triangulation of annular regions. In *Information Technology-New Generations: 15th International Conference on Information Technology*, pages 683–690. Springer, 2018.
- [7] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry, SCG '96*, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [8] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [9] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [10] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.

Bibliography

- [11] Micha Sharir, Adam Sheffer, and Emo Welzl. On degrees in random triangulations of point sets. In *Proceedings of the twenty-sixth annual symposium on Computational geometry*, pages 297–306, 2010.